DV INTERVIEW · GLANCE SHEET 7

Out-of-Order Transactions & IDs

How AXI lets fast responses overtake slow ones — and the single rule that keeps it safe.

Same ID → in issue order
Different ID → any order

1 What & why

- **Out-of-order (OoO)**: responses come back in a different order than the requests were issued.
- Possible because AXI has **multiple outstanding transactions** and tags each with an **ID**.
- Why it matters: slaves have very different latencies — an SRAM answers in 2 cycles, DDR in 50+. In-order would make every fast access wait behind the slow one (**head-of-line blocking**).
- DDR controllers also reorder internally (open-row hits first), so they *need* OoO to be efficient.

One-liner: IDs let the system return each response as soon as it's ready, while preserving order only where the master needs it.

2 The ordering rules

#	Rule
1	**Same ID, same direction** → responses must return in the order the addresses were issued.
2	**Different IDs** → no ordering requirement; may complete in any order.
3	Read data **may interleave** between different RIDs; beats **within one burst stay in order**.
4	Write responses (B) may return OoO for different BIDs.
5	**AXI4 write data** must follow AW order — no interleaving (AXI3 allowed it via WID).
6	**No ordering between reads and writes**, even with the same ID. Need it? Wait for the response first.

A master that always uses **one ID** gets fully in-order behaviour for free. Using several IDs is an opt-in to OoO.

3 Same request, two ID choices — A to slow DDR (4 beats), B to fast SRAM (2 beats)

DIFFERENT IDS (A = ID0, B = ID1) → OUT-OF-ORDER ALLOWED



B finishes at cycle **5**, A at **10**. B overtook A — legal because the IDs differ.

SAME ID (BOTH ID0) → MUST STAY IN ORDER



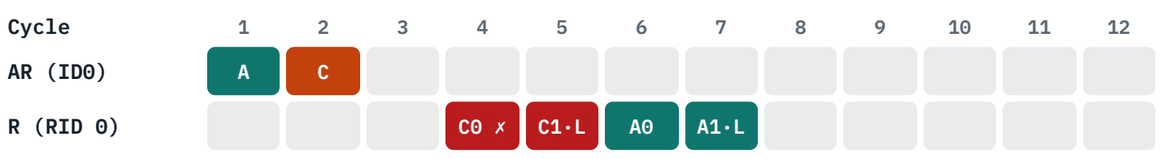
B's data was ready at cycle 4 but is **blocked behind A** until cycle 11 → head-of-line blocking.

READ DATA INTERLEAVING (DIFFERENT RIDS, BEAT LEVEL)



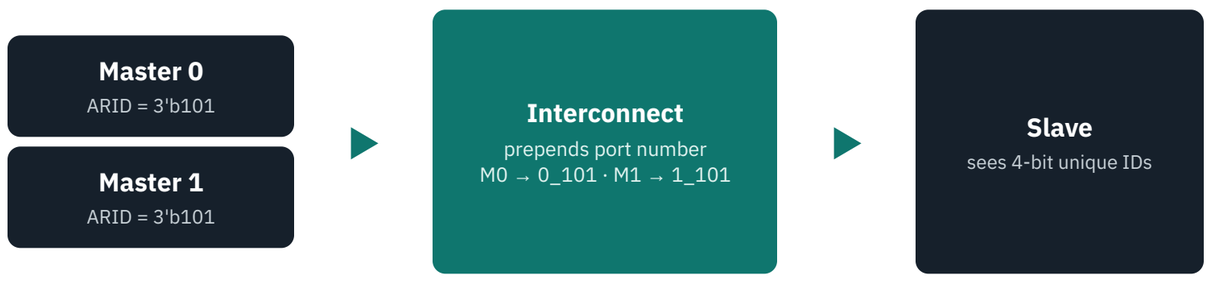
Legal: beats of A and B mixed. Illegal: A1 before A0 (order *within* a burst is fixed).

ILLEGAL: SAME ID RETURNED OUT OF ORDER



Master would think C's data belongs to A — data corruption. Scoreboards must flag this.

4 What the interconnect does with IDs



- **ID widening:** slave-side ID = master-side ID + log2(#masters) bits. Two masters can reuse the same ID without clashing.
- **Response routing:** the extra bits on RID/BID tell the interconnect which master to send the response to; it strips them on the way back.
- **Same ID to two different slaves:** the slaves can't coordinate, so the interconnect must **stall** the 2nd request until the 1st completes (or reorder responses itself). This is the "single-slave per ID" rule many fabrics use.

Interview favourite: "Master sends ID0 to slave X, then ID0 to slave Y. Y is faster — what happens?" → interconnect blocks or buffers so X's response still reaches the master first.

5 Write side & capabilities

Item	AXI3	AXI4
B responses OoO (diff BID)	Yes	Yes
W data interleaving	Yes (WID)	**No**
W data order	By WID	Same order as AW
R data interleaving	Yes	Yes

PARAMETERS TO KNOW FOR ANY DUT

- **ID width** — how many distinct streams a master can use.
- **Read / write acceptance capability** — max outstanding transactions a slave accepts.
- **Issuing capability** — max outstanding a master will issue.
- **Read data reordering depth** — how many different IDs a slave may reorder between.
- **Read interleaving depth** — how many bursts can be active on R at once (1 = no interleave).

Reordering depth 1 = slave is **in-order**. Always legal, just slower.

✓ Verifying OoO — per-ID scoreboard

```
class axi_rd_sb extends uvm_scoreboard;
  // one FIFO of expected transactions per ID
  axi_txn exp_q[bit [ID_W-1:0]][$];

  function void write_ar(axi_txn t);   // from AR monitor
    exp_q[t.id].push_back(t);
  endfunction

  function void write_r(axi_txn t);    // full burst from R monitor
    axi_txn e;
    if (!exp_q.exists(t.id) || exp_q[t.id].size() == 0)
      `uvm_error("SB", $sformatf("RID %0h not outstanding", t.id))
    else begin
      e = exp_q[t.id].pop_front();     // oldest of SAME ID
      if (!t.data_match(e)) `uvm_error("SB", "data mismatch")
    end
  endfunction
endclass
```

Popping the **front of the same-ID queue** enforces rule 1; using a queue per ID permits rule 2 automatically.

CHECKS & COVERAGE

- RID/BID always matches an outstanding ID; no response for an ID with an empty queue.
- Monitor collects R beats **per RID** so interleaved bursts rebuild correctly.
- Cover: OoO actually happened (completion order ≠ issue order), same-ID back-to-back, max outstanding, interleaved R, all ID values.
- Slave VIP knob: random response reordering + random interleaving.

? Out-of-order interview Q&A

Question	Crisp answer
What enables OoO in AXI?	Multiple outstanding transactions + ID tags.
What is the ordering rule?	Same ID in order; different IDs any order.
Can beats inside one burst reorder?	No — only whole transactions with different IDs.
Can read data interleave?	Yes, across different RIDs.
Can write data interleave in AXI4?	No — WID removed; W follows AW order.
Is a write ordered with a later read of the same ID?	No — wait for BRESP before issuing the read.
How does the response find its master?	Interconnect-added ID bits route it back.
Same ID to two slaves — risk?	Order violation; interconnect must stall or reorder.
Benefit of OoO?	Removes head-of-line blocking; better throughput and latency.
Cost of OoO?	ID tracking and reorder buffers in master/interconnect.
How do you test it?	Per-ID queue scoreboard, reordering slave VIP, coverage on completion order.